

AI-TECH

Przykład 5 DO LABORATORIUM 1 z UCZENIA MASZYNOWEGO

Regresja liniowa i logistyczna

Tomasz Neumann

Regresja logistyczna w StatsModels

Projekt współfinansowany ze środków Unii Europejskiej w ramach Europejskiego Funduszu Rozwoju Regionalnego Program Operacyjny Polska Cyfrowa na lata 2014-2020,
Oś Priorytetowa nr 3 "Cyfrowe kompetencje społeczeństwa" Działanie nr 3.2
"Innowacyjne rozwiązania na rzecz aktywizacji cyfrowej" Tytuł projektu: „Akademia
Innowacyjnych Zastosowań Technologii Cyfrowych (AI Tech)”

1. Przykład 5

Regresja logistyczna może być także analizowana w Pythonie za pomocą pakietu StatsModels. Procedura postępowania jest podobna jak w *scikit-learn*, jednak pakiet ten oferuje więcej szczegółowych informacji statystycznych dotyczących modeli i wyników. Na samym początku należy rozpocząć od zaimportowania biblioteki NumPy oraz *statsmodels.api*.

```
In [2]: # Import podstawowych bibliotek
import numpy as np
import statsmodels.api as sm
```

Tak jak poprzednio, na potrzeby tego przykładu stworzymy tablice NumPy równomiernie rozłożonych wartości wejściowych x i wyjściowych y . Należy jednak pamiętać, że pakiet StatsModels nie bierze pod uwagę przecięcia (wyrazu wolnego) i należy uwzględnić dodatkową kolumnę 'jedynek' w x (obciążenie), co w prosty sposób można dokonać za pomocą *add_constant()*.

```
In [3]: # Stworzenie przykładowych danych

# Dwuwymiarowa tablica danych wejściowych:
# jedna kolumna dla jednego wejścia, liczba wierszy równa liczbie obserwacji
x = np.arange(10).reshape(-1, 1)
y = np.array([0, 1, 0, 0, 1, 1, 1, 1, 1, 1])
#dodanie kolumny z jedynekami
x = sm.add_constant(x)

#wyświetlenie kontrolne utworzonych danych
print(x)
print(y)
```

```
[[1. 0.]
 [1. 1.]
 [1. 2.]
 [1. 3.]
 [1. 4.]
 [1. 5.]
 [1. 6.]
 [1. 7.]
 [1. 8.]
 [1. 9.]]
[0 1 0 0 1 1 1 1 1 1]
```

Zbudowanie modelu klasyfikacji nastąpi za pomocą klasy `statsmodels.discrete.discrete_model.Logit`. Należy tutaj pamiętać, że pierwszym argumentem są tu wartości zmiennej zależnej y a następnie zmiennej x .
https://www.statsmodels.org/stable/generated/statsmodels.discrete.discrete_model.Logit.html
 Trenowanie modelu realizowane jest za pomocą `.fit()` lub jeżeli ma być wykorzystywana regularyzacja L1, za pomocą metody `.fit_regularized()`.



```
In [4]: # Zbudowanie modelu klasyfikacji za pomocą klasy LogisticRegression
# https://www.statsmodels.org/stable/generated/statsmodels.discrete.discrete_mod
model = sm.Logit(y, x)
# wytrenowanie modelu na podstawie wcześniej wygenerowanych danych
result = model.fit(method='newton')
#Odkomentuj poniżej trzy linijki, jakby pojawiły się jakieś ostrzeżenia
#import warnings
#from statsmodels.tools.sm_exceptions import ConvergenceWarning
#warnings.simplefilter('ignore', ConvergenceWarning)
result.params
```

```
Optimization terminated successfully.
      Current function value: 0.350471
      Iterations 7
```

```
Out[4]: array([-1.972805 ,  0.82240094])
```

Aby sprawdzić wytrenowany model i uzyskać wartość szansy przynależności do danej klasy dla analizowanej obserwacji można użyć metody `.predict()`.

```
In [5]: # Wyświetlenie wyliczonych z modelu prawdopodobieństw
print('Wyznaczone szanse przynależności do danej klasy ("prawdopodobieństwa"):')
print(result.predict(x))
# Przydzielenie do danej klasy z wartością progową 0.5
print("Przynależność do klas:")
print((result.predict(x) >= 0.5).astype(int))
```

```
Wyznaczone szanse przynależności do danej klasy ("prawdopodobieństwa"):
[0.12208792 0.24041529 0.41872657 0.62114189 0.78864861 0.89465521
 0.95080891 0.97777369 0.99011108 0.99563083]
Przynależność do klas:
[0 0 0 1 1 1 1 1 1 1]
```

Wartości, które zostały tu uzyskane można porównać z tymi jakie można uzyskać z wykorzystaniem biblioteki scikit-learn - najprawdopodobniej będą się one różnić od siebie, co jest spowodowane zastosowaniem różnych metod iteracyjnych i procedur oraz innych ustawień parametrów wejściowych.

W celu uzyskania macierzy pomyłek z wytrenowanego modelu można użyć metody `.pred_table()`, które wartości są typu zmiennoprzecinkowego.

```
In [6]: print(result.pred_table())
```

```
[[2. 1.]
 [1. 6.]]
```

Do uzyskania bardziej szczegółowych statystyk służy metoda `summary()`

```
In [7]: print(result.summary())
```

```

                                Logit Regression Results
=====
Dep. Variable:                  y      No. Observations:                  10
Model:                          Logit   Df Residuals:                      8
Method:                        MLE     Df Model:                          1
Date:                          Sat, 26 Mar 2022   Pseudo R-squ.:                  0.4263
Time:                          16:14:50   Log-Likelihood:                 -3.5047
converged:                      True    LL-Null:                        -6.1086
Covariance Type:                nonrobust   LLR p-value:                    0.02248
=====
               coef      std err          z      P>|z|      [0.025      0.975]
-----
const        -1.9728        1.737       -1.136      0.256      -5.377       1.431
x1             0.8224        0.528        1.557      0.119      -0.213       1.858
=====

```

bądź metoda `summary2()`

```
In [8]: print(result.summary2())
```

```

                                Results: Logit
=====
Model:                          Logit      Pseudo R-squared: 0.426
Dependent Variable: y            AIC:      11.0094
Date:                          2022-03-26 16:14 BIC:      11.6146
No. Observations: 10            Log-Likelihood: -3.5047
Df Model: 1                    LL-Null:      -6.1086
Df Residuals: 8                LLR p-value:  0.022485
Converged: 1.0000              Scale:      1.0000
No. Iterations: 7.0000
=====
               Coef.      Std.Err.          z      P>|z|      [0.025      0.975]
-----
const        -1.9728        1.7366       -1.1360     0.2560     -5.3765     1.4309
x1             0.8224        0.5281        1.5572     0.1194     -0.2127     1.8575
=====

```

Zadania do wykonania

Zadanie 1

Wczytaj dane z pliku *diabetes.csv*, wykonaj odpowiednie dopasowanie danych do modelu korzystając z powyższego pakietu oraz przeprowadź przykładową klasyfikację. (W pliku *diabetes.csv* znajdują się informacje na temat: liczby cięż, stężenia glukozy w osoczu, ciśnienia rozkurczowego krwi, grubości fałdu mięśnia trójgłowego, insuliny w surowicy krwi, wskaźnika BMI, funkcji powstania cukrzycy na podstawie wywiadu rodzinnego, informację o wieku oraz jako ostatnią wartość informującą czy dana osoba nie ma cukrzycy - '0', czy też ma cukrzycę - '1'.

